LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 11. Tuple

Lecture: 123. Indexing & slicing - Tuple

Section 1: Lecture Summary

This lecture revisits the concepts of indexing and slicing, specifically applied to tuples in Python. While students may already be familiar with these topics from work with strings and lists, the lecture highlights how the same principles apply to tuples, emphasizing tuples' immutability which restricts operations to reading only. The session covers positive and negative indexing, how to access tuple elements by index, and then explores slicing with start, stop, and step parameters—including the use of negative steps to reverse the tuple. Practical examples are demonstrated in a Jupyter Notebook environment, showing the effects of various slicing patterns. The lecture concludes by underlining the importance of mastering indexing and slicing for effective data handling in Python and encourages practice.

Section 2: Key Concepts and Explanations

- Tuples:
  - Immutable sequences in Python used to store ordered collections of items.
  - Unlike lists, tuples do not allow modification after creation (no writing, only reading).

- Indexing:
  - Accessing individual elements in a tuple using an index.
  - Positive indexing starts at 0 (the first element).
  - Negative indexing starts at -1 (the last element) and goes backward.
  - Example: `t1[4]` and `t1[-3]` both access the same element.

- Slicing:
  - Extracting a subsequence (slice) from a tuple.
  - General syntax: `tuple[start:stop:step]`
    - `start`: index at which to start (inclusive).
    - `stop`: index at which to end (exclusive).
    - `step`: the stride or jump between elements (default is 1).
  - All three parameters are optional:
    - Omitting `start` defaults to 0.
    - Omitting `stop` defaults to the end of the tuple.
    - Omitting `step` defaults to 1 (forward traversal).
  - Positive indices select slices in the forward direction.
  - Negative indices can be used in start and stop to slice relative to the end.
  - Negative `step` reverses the direction of slicing, enabling backward traversal.
    - When `step` is negative, `start` must be greater than `stop` in index terms.

- Important notes:
  - Slicing with forward step excludes the stop index.
  - When slicing backwards (negative step), indexing bounds must correspond to a descending pattern.
  - Indexing and slicing in tuples are read-only operations due to immutability.

Section 3: Example Code and Use Cases

Creating a tuple with multiples of 3

Indexing examples

Slicing examples

Full slice (same as the whole tuple)

Slice from index 2 to end

Slice from index 2 to 5 (stop index excluded)

Slice using negative indices

Slicing with step: every 2nd element

Slicing with step: reverse entire tuple

Slicing with reverse step and start index 4 going backwards to 0 (stop excluded)

Equivalent slice using negative indices (reverse step)

```
t1 = (3, 6, 9, 12, 15, 18, 21)

print(t1[4])    # Output: 15 (5th element, positive index)
print(t1[-3])   # Output: 15 (3rd from last element, negative index)


print(t1[:])        # Output: (3, 6, 9, 12, 15, 18, 21)

print(t1[2:])       # Output: (9, 12, 15, 18, 21)

print(t1[2:5])      # Output: (9, 12, 15)

print(t1[-5:-2])    # Output: (9, 12, 15)

print(t1[::2])      # Output: (3, 9, 15, 21)

print(t1[::-1])     # Output: (21, 18, 15, 12, 9, 6, 3)

print(t1[4:0:-1])   # Output: (15, 12, 9, 6)

print(t1[-3:-7:-1]) # Output: (15, 12, 9, 6)
```

Explanations:

- `t1[4]` and `t1[-3]` access the same element (15).

- `t1[:]` returns the whole tuple (a shallow copy).

- `t1[2:]` returns all elements from index 2 to end.

- `t1[2:5]` selects elements with indices 2, 3, and 4.

- Using negative indices `t1[-5:-2]` selects the same elements as positive indices `2:5`.

- Using `step=2` in `t1[::2]` selects every other element starting from index 0.

- `t1[::-1]` reverses the tuple.

- When using negative `step`, the indices must reflect a reverse traversal (start > stop).

Section 4: Key Takeaways

- Tuples are immutable sequences; indexing and slicing can only read data, not modify it.

- Indexing accesses single elements by position, with positive and negative indices available.

- Slicing extracts sub-tuples with optional start, stop, and step parameters.

- Default slicing traverses forward (`step=1`); specifying a negative step (`-1`) reverses the order.

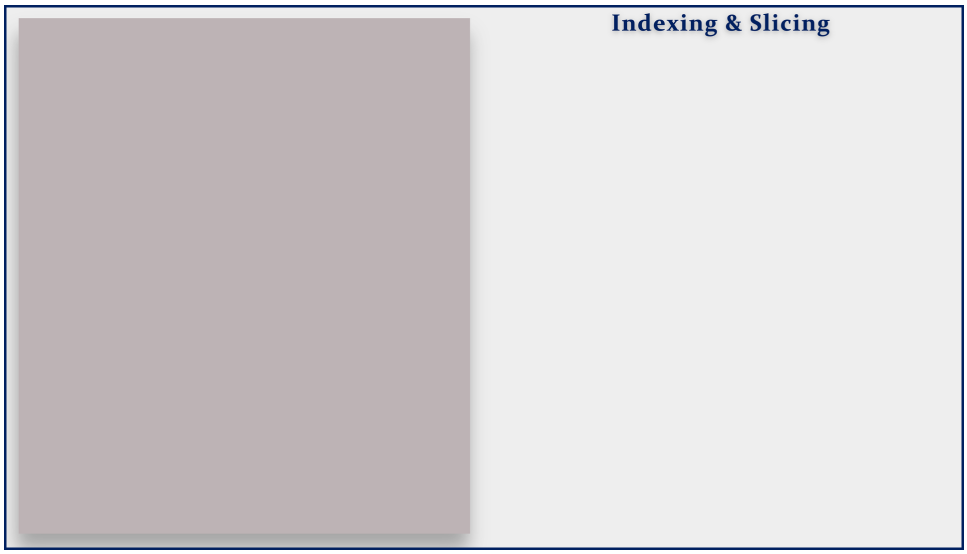
- When slicing backwards, provide indices in decreasing order (start > stop).

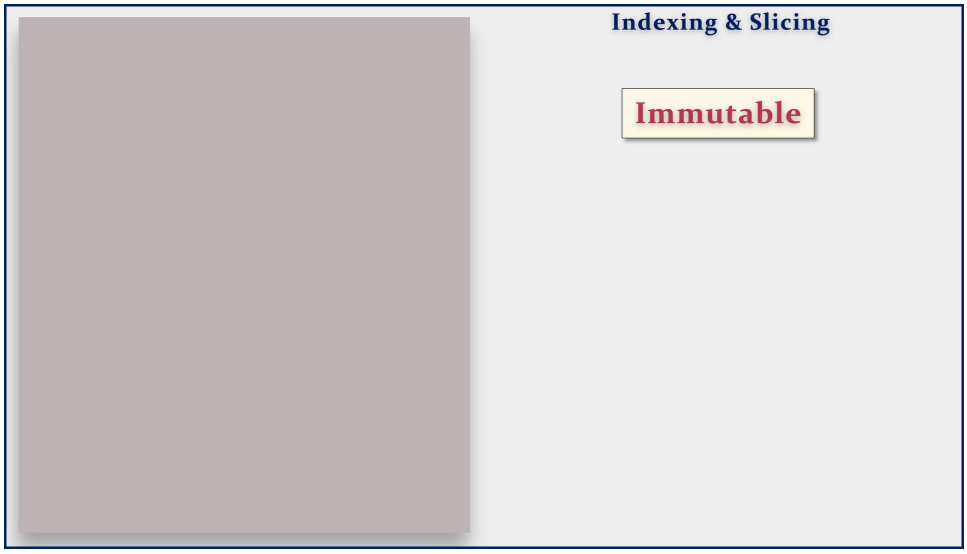
- Indexing and slicing use zero-based indexing and the stop index is always exclusive.

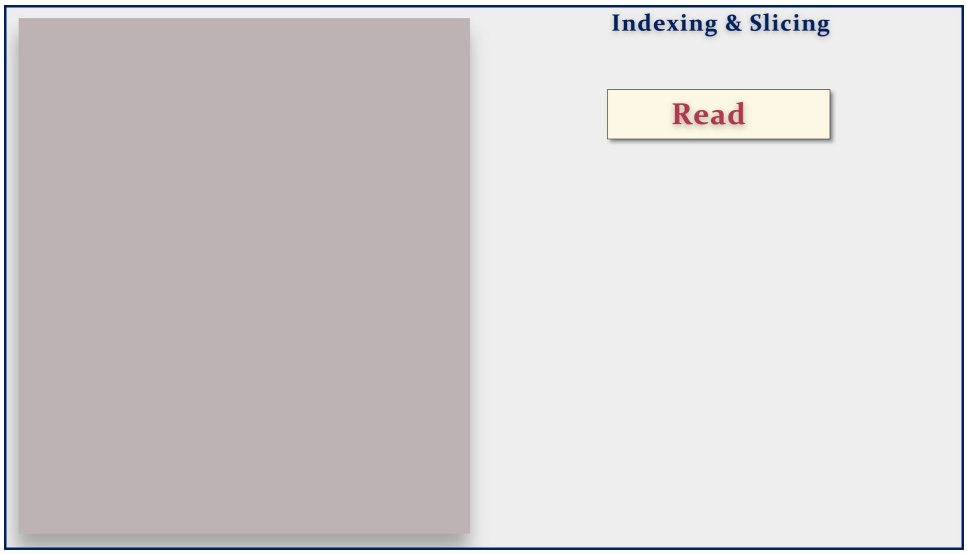
- Mastery of indexing and slicing is fundamental for efficient Python programming and data manipulation.

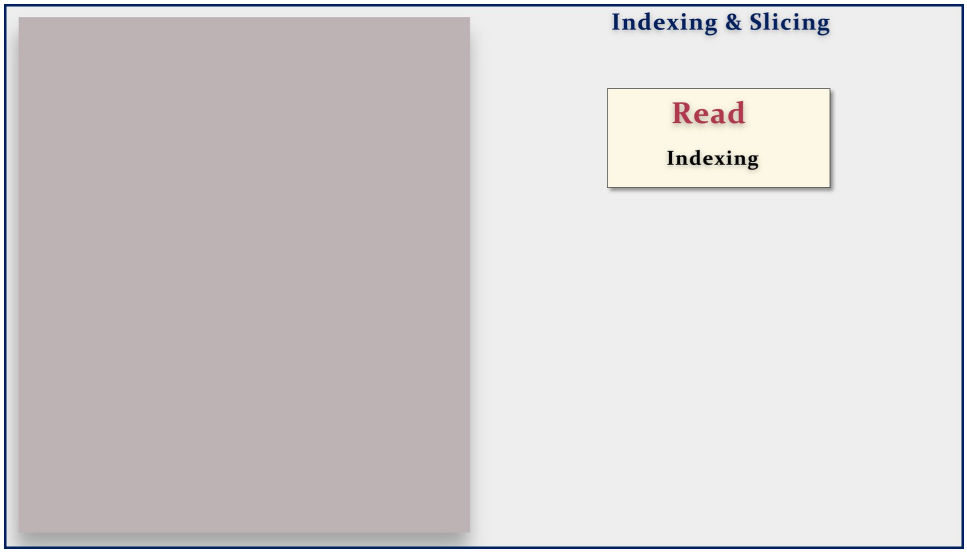
Lecture in Slides

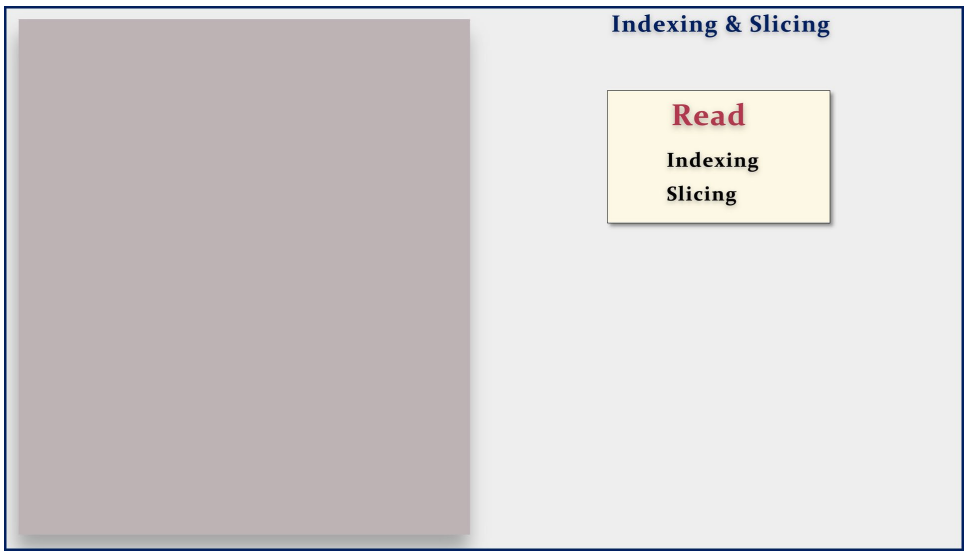
Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.

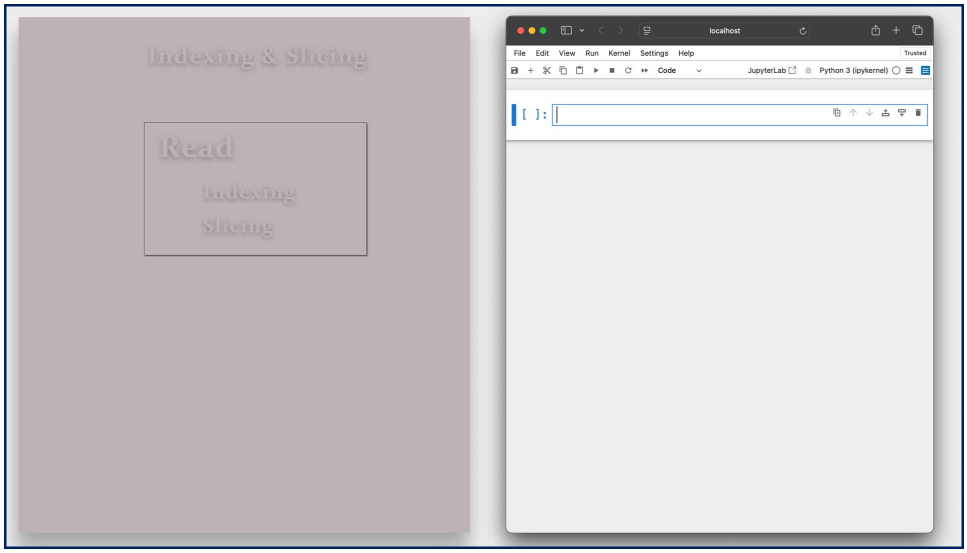

Indexing & Slicing

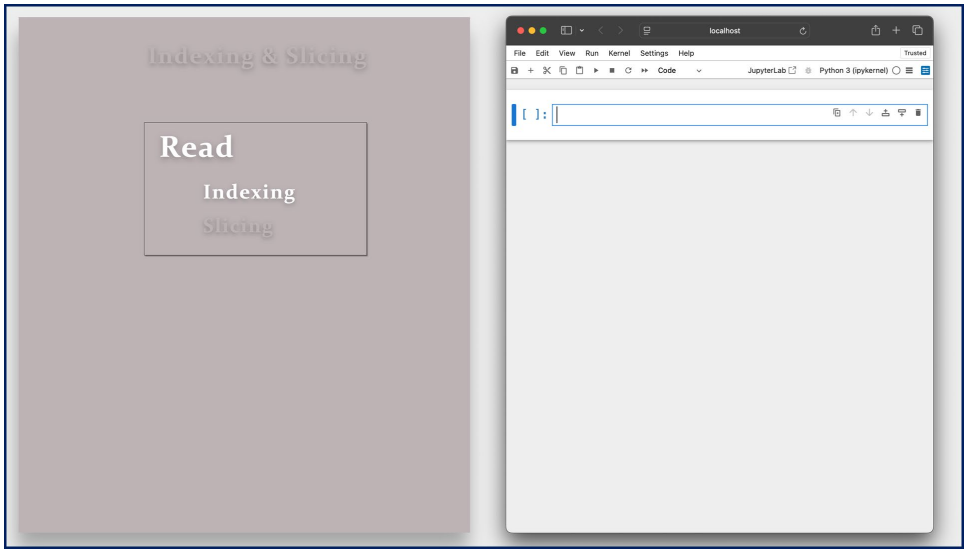

Indexing & Slicing
Immutable

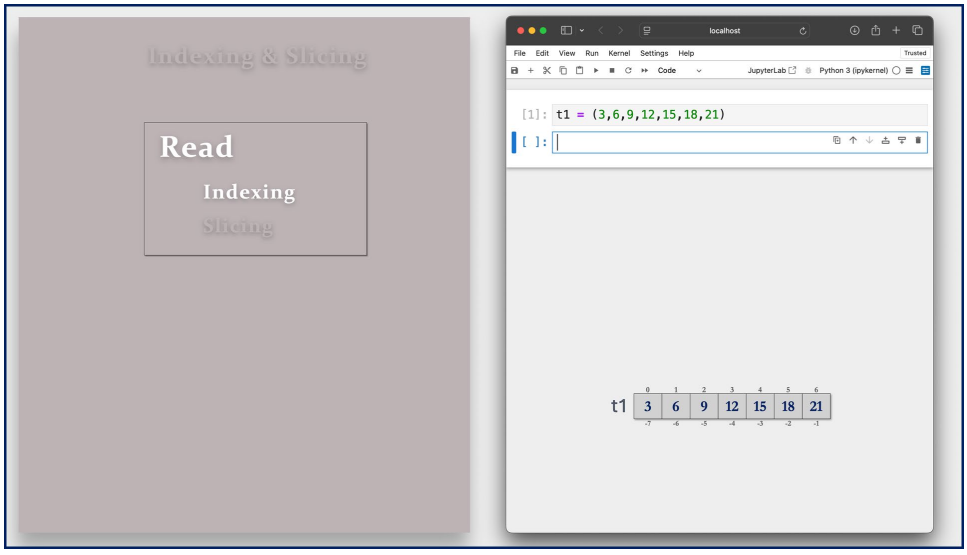

Indexing & Slicing
Read

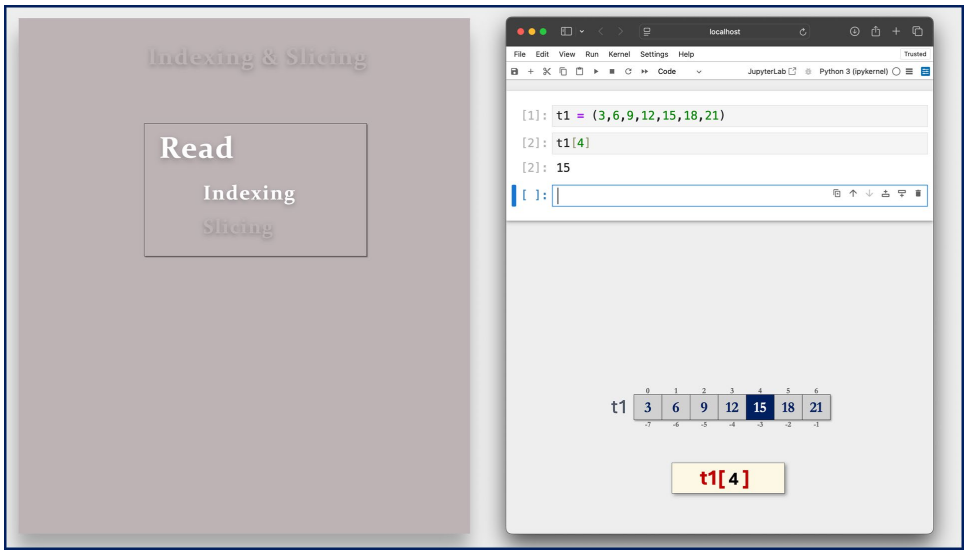

Indexing & Slicing
Read
Indexing

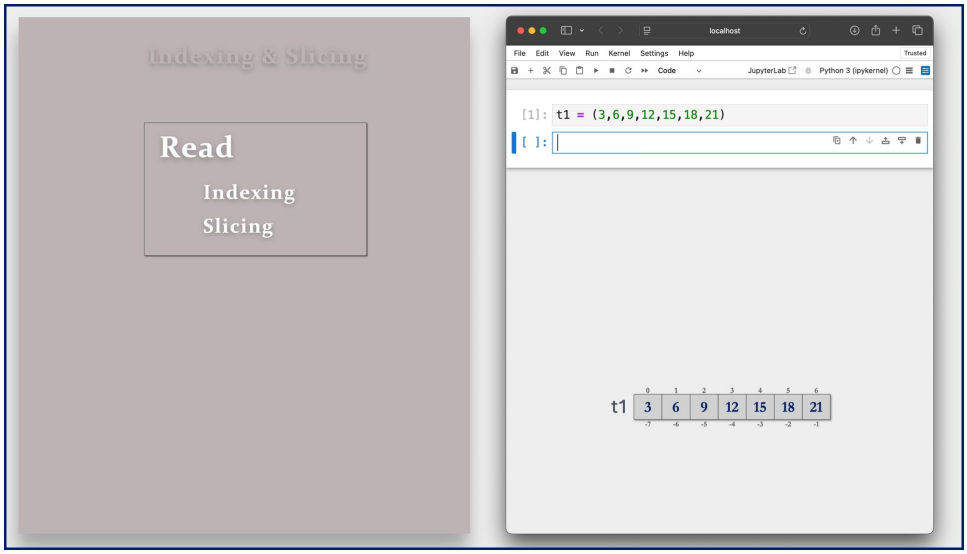

Indexing & Slicing
Read
Indexing
Slicing

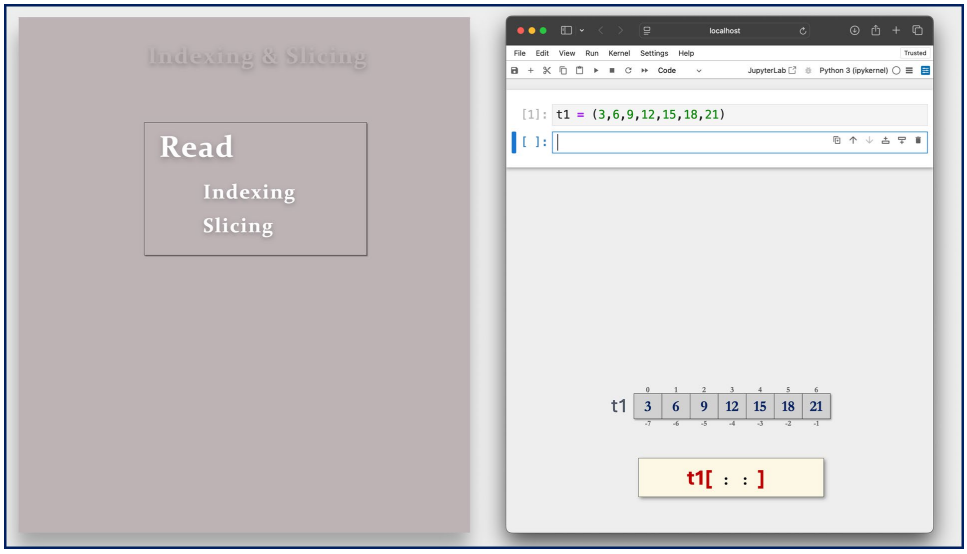

Indexing & Slicing
Read
Indexing
Slicing
localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[ ]:

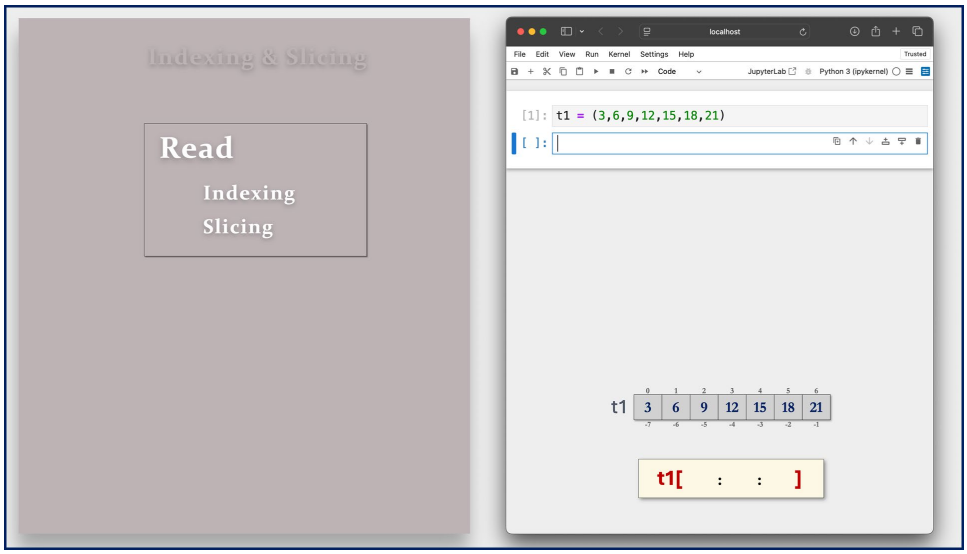

Indexing & Slicing
Read
Indexing
Slicing
localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[ ]:

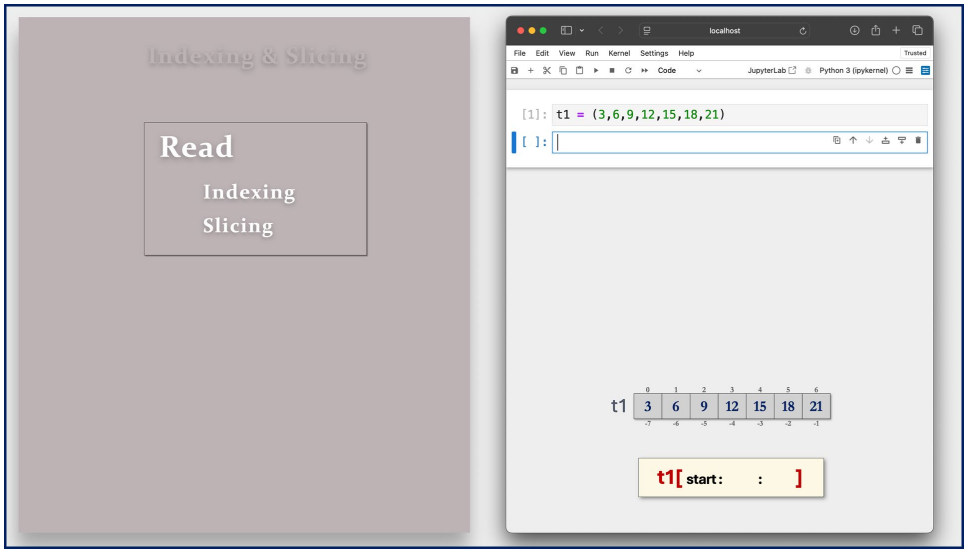

Indexing & Slicing
Read
Indexing
Slicing
localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: t1 = (3,6,9,12,15,18,21)
[ ]:
t1
0 1 2 3 4 5 6
3 6 9 12 15 18 21
-7 -6 -5 -4 -3 -2 -1

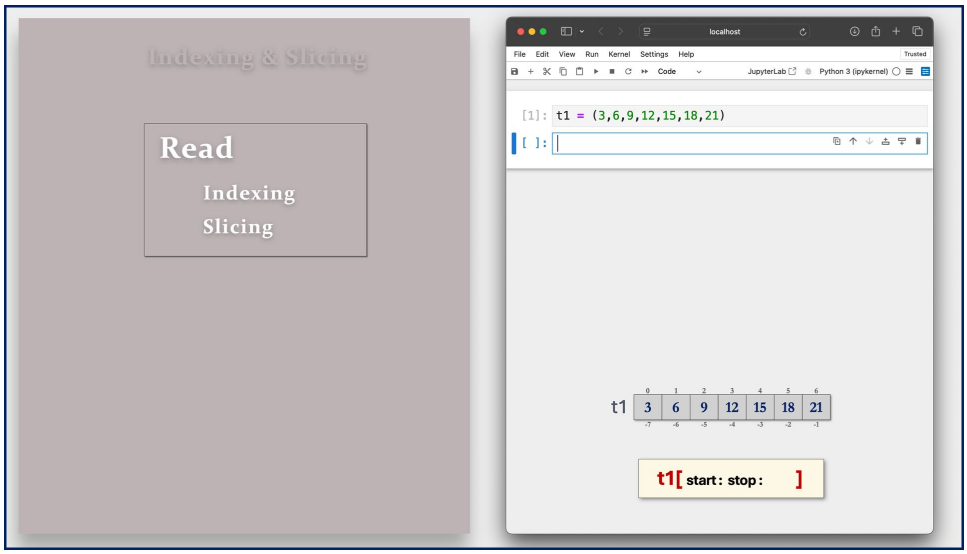

Indexing & Slicing
Read
Indexing
Slicing
localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: t1 = (3,6,9,12,15,18,21)
[2]: t1[4]
[2]: 15
[ ]:
t1
0 1 2 3 4 5 6
3 6 9 12 15 18 21
-7 -6 -5 -4 -3 -2 -1
t1[4]

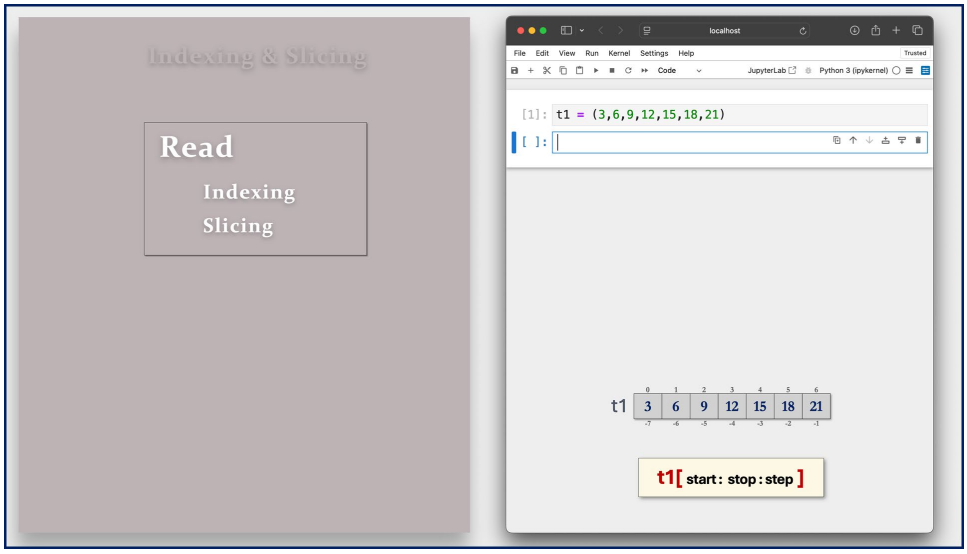

Indexing & Slicing
Read
Indexing
Slicing
localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: t1 = (3,6,9,12,15,18,21)
[ ]:
t1
0 1 2 3 4 5 6
3 6 9 12 15 18 21
-7 -6 -5 -4 -3 -2 -1

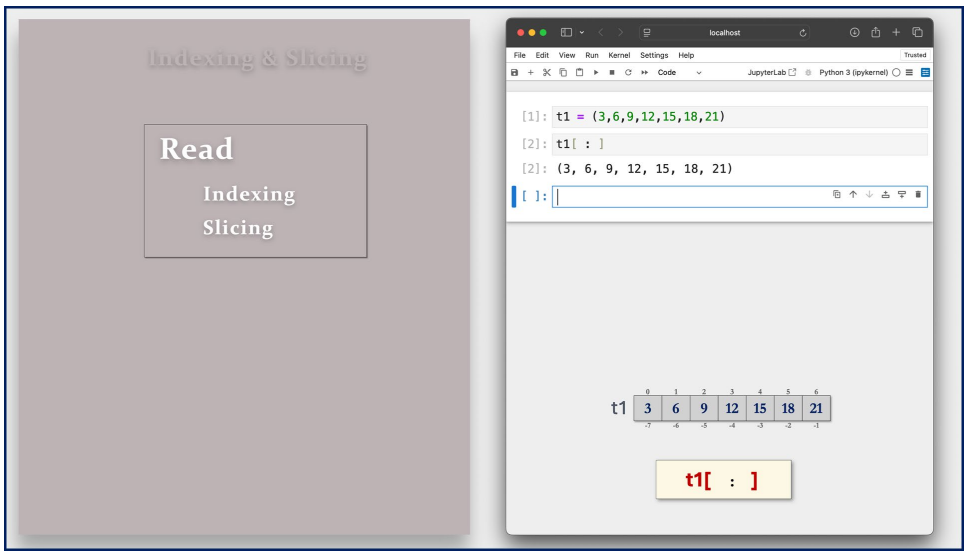

Indexing & Slicing
Read
Indexing
Slicing
localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: t1 = (3,6,9,12,15,18,21)
[ ]:
t1
0 1 2 3 4 5 6
3 6 9 12 15 18 21
-7 -6 -5 -4 -3 -2 -1
t1[: :]

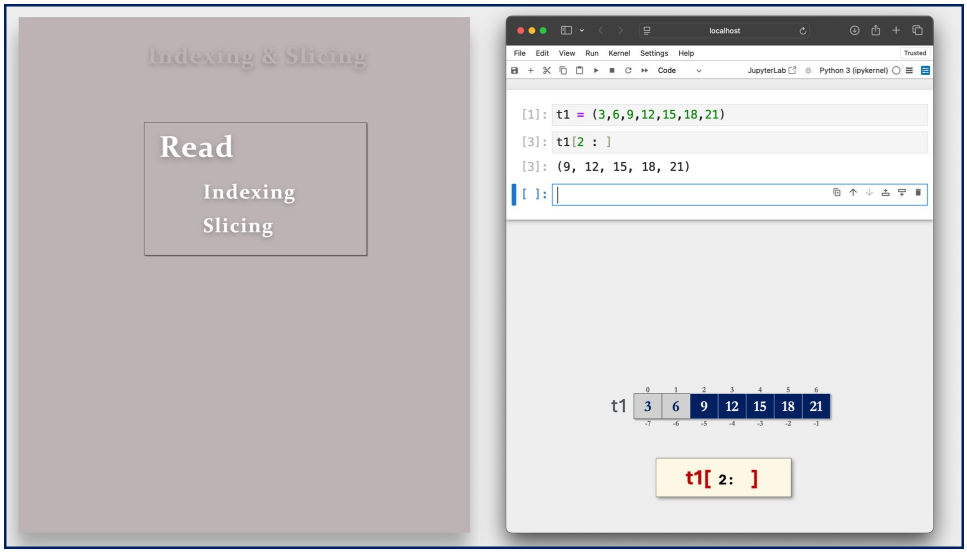

Indexing & Slicing
Read
Indexing
Slicing
localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: t1 = (3,6,9,12,15,18,21)
[ ]:
t1
0 1 2 3 4 5 6
3 6 9 12 15 18 21
-7 -6 -5 -4 -3 -2 -1
t1[: :]

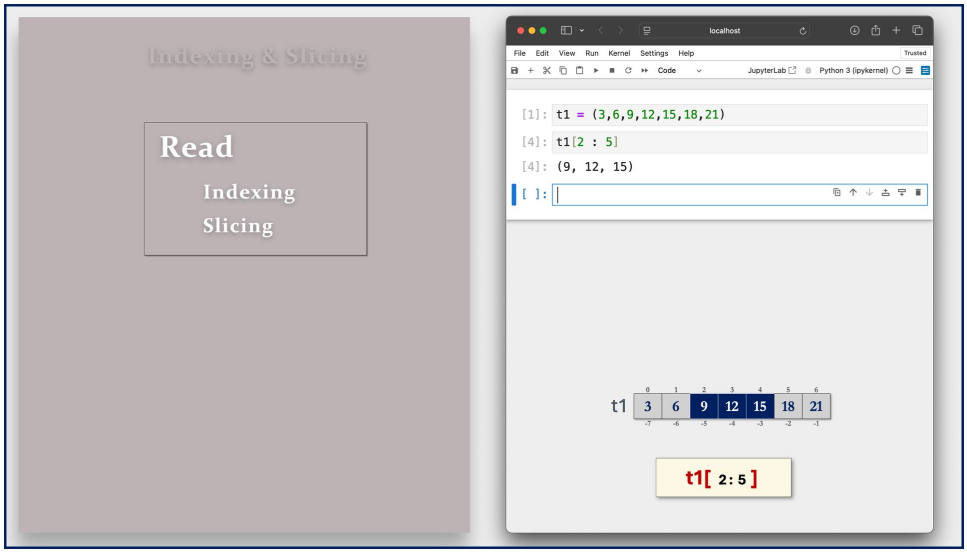

Indexing & Slicing
Read
Indexing
Slicing
localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: t1 = (3,6,9,12,15,18,21)
[ ]:
t1
0 1 2 3 4 5 6
3 6 9 12 15 18 21
-7 -6 -5 -4 -3 -2 -1
t1[start : :]

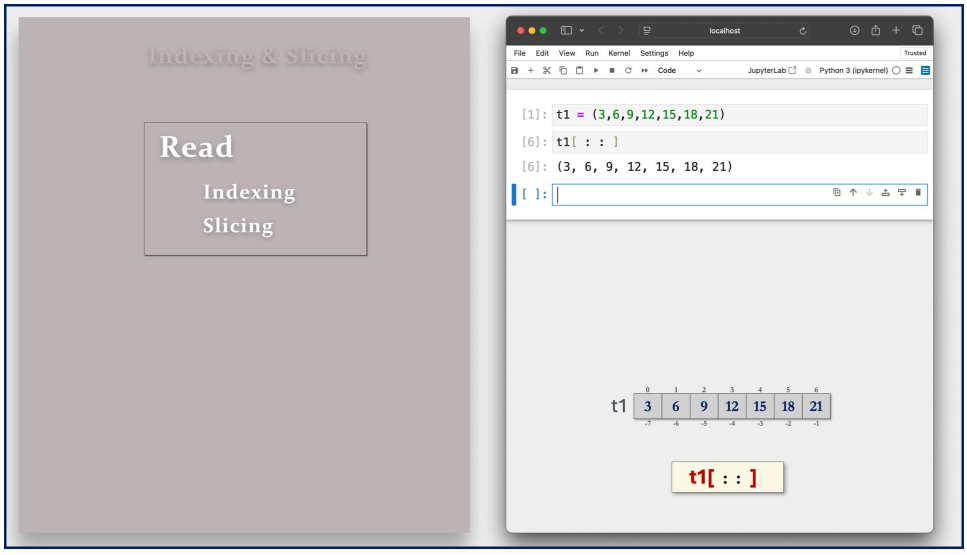

Indexing & Slicing
Read
Indexing
Slicing
localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: t1 = (3,6,9,12,15,18,21)
[ ]:
t1
0 1 2 3 4 5 6
3 6 9 12 15 18 21
-7 -6 -5 -4 -3 -2 -1
t1[start : stop :]

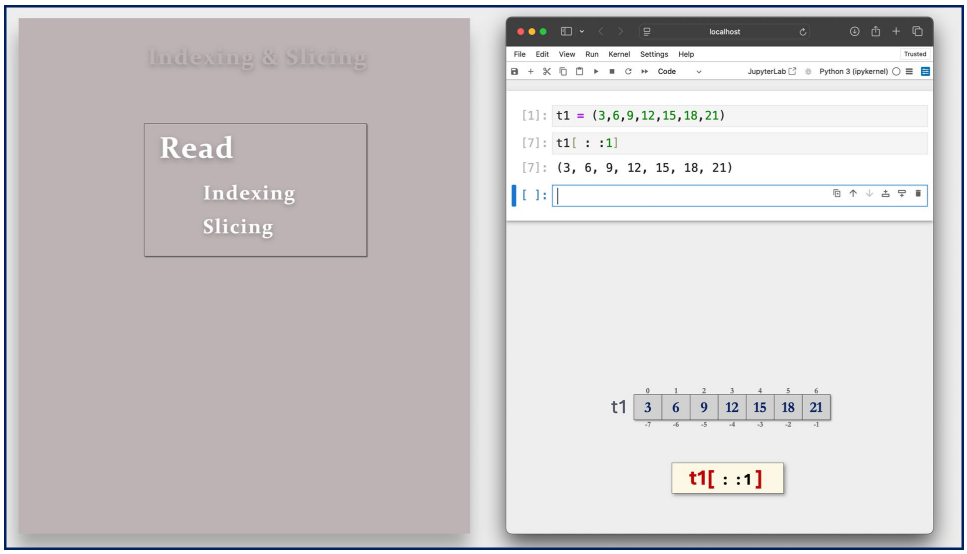

Indexing & Slicing
Read
Indexing
Slicing
localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: t1 = (3,6,9,12,15,18,21)
[ ]:
t1
0 1 2 3 4 5 6
3 6 9 12 15 18 21
-7 -6 -5 -4 -3 -2 -1
t1[start : stop : step]

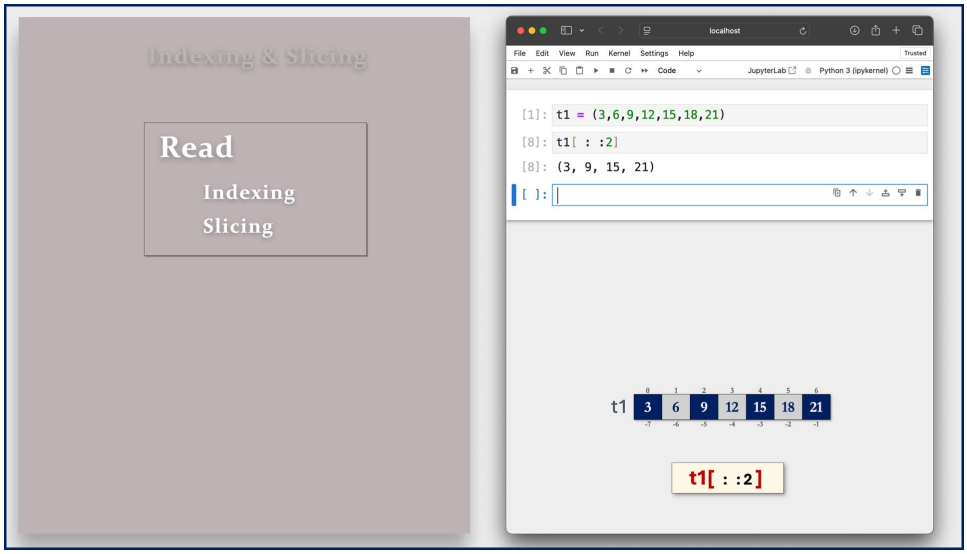

Indexing & Slicing
Read
Indexing
Slicing
localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: t1 = (3,6,9,12,15,18,21)
[2]: t1[:]
[2]: (3, 6, 9, 12, 15, 18, 21)
[ ]:
t1
3 6 9 12 15 18 21
t1[:]

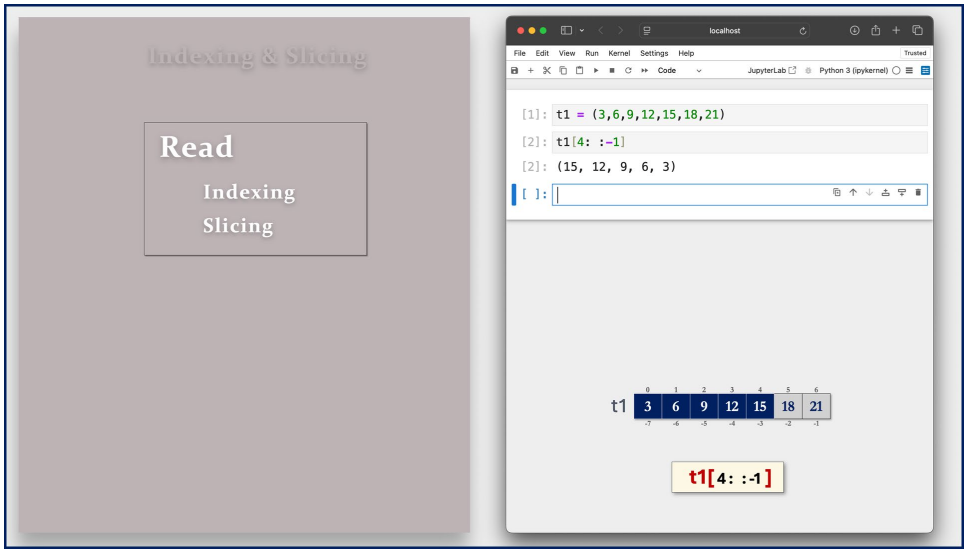

Indexing & Slicing
Read
Indexing
Slicing
localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: t1 = (3,6,9,12,15,18,21)
[3]: t1[2 :]
[3]: (9, 12, 15, 18, 21)
[ ]:
t1
3 6 9 12 15 18 21
t1[2:]

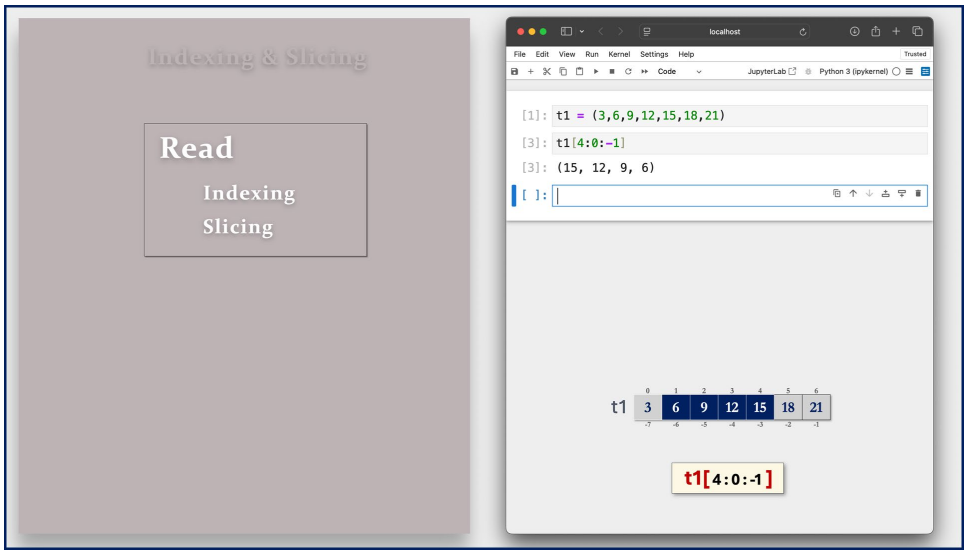

Indexing & Slicing
Read
Indexing
Slicing
localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: t1 = (3,6,9,12,15,18,21)
[4]: t1[2 : 5]
[4]: (9, 12, 15)
[ ]:
t1
3 6 9 12 15 18 21
t1[2:5]


Indexing & Slicing
Read
Indexing
Slicing
localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: t1 = (3,6,9,12,15,18,21)
[6]: t1[: :]
[6]: (3, 6, 9, 12, 15, 18, 21)
[ ]:
t1
3 6 9 12 15 18 21
t1[: :]


Indexing & Slicing
Read
Indexing
Slicing
localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: t1 = (3,6,9,12,15,18,21)
[7]: t1[: :1]
[7]: (3, 6, 9, 12, 15, 18, 21)
[ ]:
t1
3 6 9 12 15 18 21
t1[: :1]


Indexing & Slicing
Read
Indexing
Slicing
localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: t1 = (3,6,9,12,15,18,21)
[8]: t1[: :2]
[8]: (3, 9, 15, 21)
[ ]:
t1
3 6 9 12 15 18 21
t1[: :2]


Indexing & Slicing
Read
Indexing
Slicing
localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: t1 = (3,6,9,12,15,18,21)
[2]: t1[4: :-1]
[2]: (15, 12, 9, 6, 3)
[ ]:
t1
3 6 9 12 15 18 21
t1[4: :-1]


Indexing & Slicing
Read
Indexing
Slicing
localhost
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]: t1 = (3,6,9,12,15,18,21)
[3]: t1[4:0:-1]
[3]: (15, 12, 9, 6)
[ ]:
t1
0 1 2 3 4 5 6
3 6 9 12 15 18 21
-7 -6 -5 -4 -3 -2 -1
t1[4:0:-1]